

ek

bas

Co

ek

cd1
282

bas

Co

kubectl get nodes

```
cdickersoncloudcoder@BeastNode: /mnt/c/Users/19168/Desktop$ kubectl get nodes
NAME                                STATUS    ROLES    AGE   VERSION
ip-10-0-1-10.internal               Ready    <none>   3m39s v1.27.16-eks-aac579
cdickersoncloudcoder@BeastNode: /mnt/c/Users/19168/Desktop$
```

💡 This verifies the status of the nodes in your EKS cluster to ensure they are up and running.

📁 AWS EC2 Console (Instance Details): [EC2 Instance Details](#)

Instance summary for i-**[redacted]** (demo-cluster-demo-nodes-Node) info

Updated less than a minute ago

Instance ID: **[redacted]**

IPv6 address: -

Hostname type: IP name: **[redacted]**.internal

Answer private resource DNS name: -

Auto-assigned IP address: **[redacted]** (Public IP)

IAM Role: **[redacted]** (Circled in blue)

IMDSv2: Required

Operator: -

Public IPv4 address: **[redacted]** | open address

Instance state: **Running**

Private IP DNS name (IPv4 only): **[redacted]**

Instance type: t2.micro

VPC ID: **[redacted]** (demo-cluster-cluster/VPC)

Subnet ID: **[redacted]** (demo-cluster-SubnetPublicUSEAST1F)

Instance ARN: **[redacted]**

Private IPv4 addresses: **[redacted]**

Public IPv4 DNS: **[redacted]**.compute-1.amazonaws.com | open address

Elastic IP addresses: -

AWS Compute Optimizer finding: Opt-in to AWS Compute Optimizer for recommendations. | Learn more

Auto Scaling Group name: **[redacted]**

Managed: false

💡 This link shows the EC2 instance related to the EKS cluster for management and monitoring.

📁 AWS CloudFormation Console: [CloudFormation Stacks](#)

Stacks (2)

Filter by stack name:

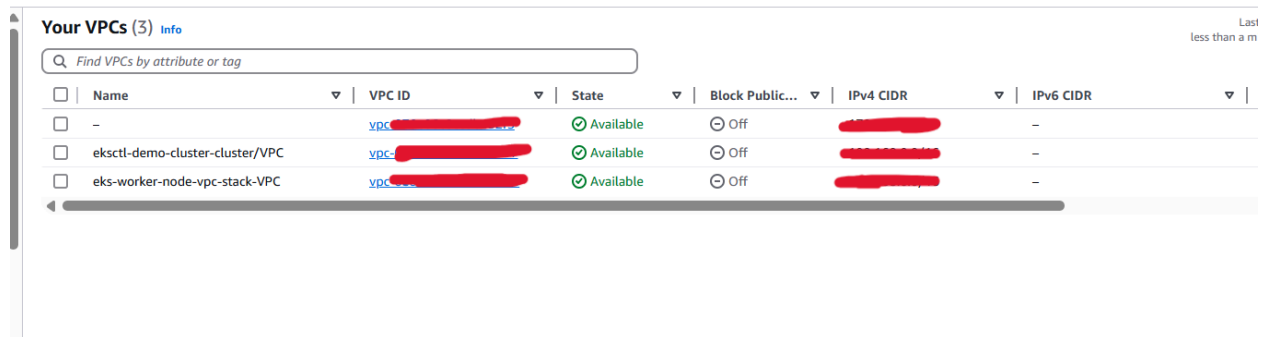
Filter status: **Active**

☐ View nested

Stack name	Status	Created time	Description
eksctl-demo-cluster-nodegroup-demo-nodes	CREATE_COMPLETE	2025-05-11 08:38:22 UTC-0700	EKS Managed Nodes (SSH access: false) [created by eksctl]

💡 This link provides an overview of the CloudFormation stack that is created when the EKS cluster is launched, showing the resources provisioned.

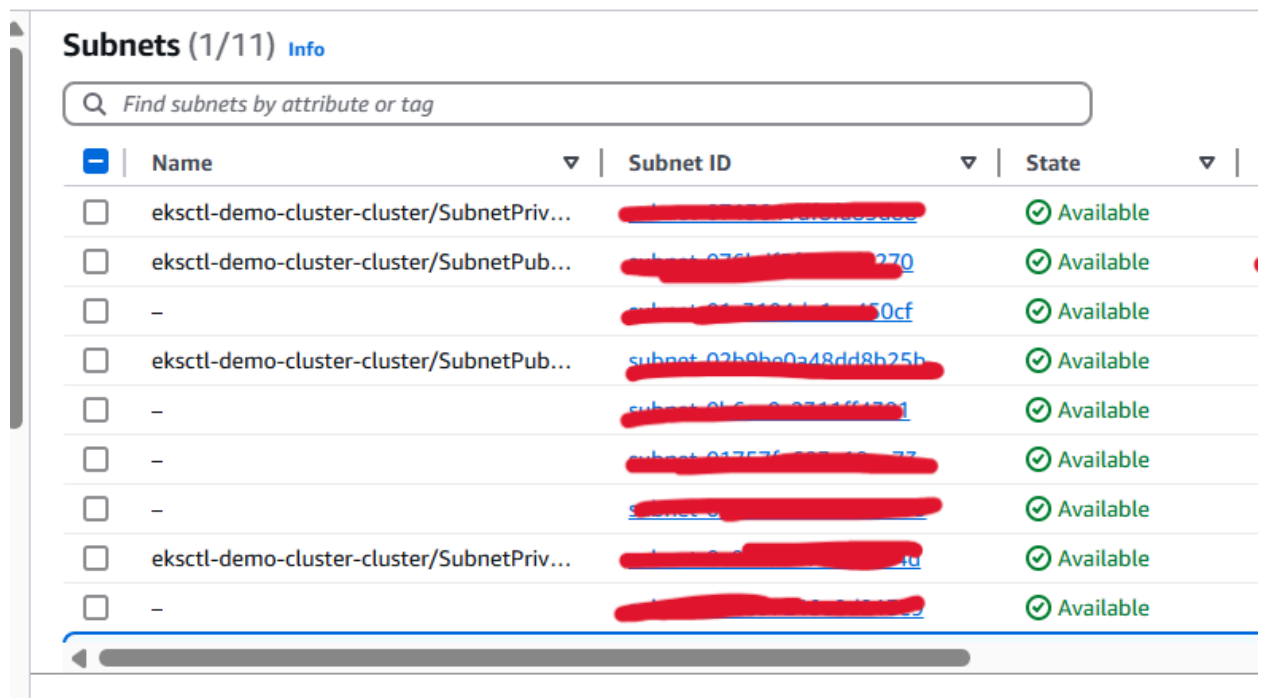
📁 **AWS VPC Console (VPC Overview):**
[VPC Dashboard](#)



☐	Name	VPC ID	State	Block Public...	IPv4 CIDR	IPv6 CIDR
☐	-	vpc-...	Available	Off	...	-
☐	eksctl-demo-cluster-cluster/VPC	vpc-...	Available	Off	...	-
☐	eks-worker-node-vpc-stack-VPC	vpc-...	Available	Off	...	-

💡 This link directs you to the VPC settings for the created EKS cluster, where you can review the network configurations.

📁 **AWS VPC Console (Subnets Overview):**
[VPC Subnets](#)



☐	Name	Subnet ID	State
☐	eksctl-demo-cluster-cluster/SubnetPriv...	subnet-...	Available
☐	eksctl-demo-cluster-cluster/SubnetPub...	subnet-...	Available
☐	-	subnet-...	Available
☐	eksctl-demo-cluster-cluster/SubnetPub...	subnet-...	Available
☐	-	subnet-...	Available
☐	-	subnet-...	Available
☐	-	subnet-...	Available
☐	eksctl-demo-cluster-cluster/SubnetPriv...	subnet-...	Available
☐	-	subnet-...	Available

💡 This link shows the subnets created in your VPC that are associated with the EKS cluster.

📁 **AWS EKS Console (Cluster Overview):**
[EKS Cluster Console](#)

Clusters (1) [Info](#)

Filter clusters

	Cluster name	Status	Kubernetes version	Support period	Upgrade policy
☐	demo-cluster	Active	1.27 Upgrade now	Extended support until July 23, 2025	Extended

💡 This link takes you to the EKS console where you can view the cluster status, configuration, and other essential details.

🔍 What We Learned:

- We successfully created an EKS cluster using `eksctl` and verified the status of the nodes in Kubernetes.
- We gained an understanding of how to navigate the AWS Console to monitor and manage the resources tied to our EKS cluster.
- We familiarized ourselves with the EC2 instances, CloudFormation stacks, VPC settings, and subnets related to the EKS cluster.

✅ Summary:

You've just created and managed an EKS cluster, gaining valuable insights into how to configure and monitor Kubernetes clusters in AWS. This adds to your understanding of cloud infrastructure by showing how to deploy, inspect, and manage resources effectively using AWS services.

📁 Step 2: 🔧 Configuring Kubernetes Access for Your Amazon EKS Cluster

◆ Step Overview:

In this step, we configured access to the Amazon EKS cluster using `aws eks update-kubeconfig`. This allowed us to interact with the Kubernetes API server, making it possible to manage the cluster directly through `kubectl`.

⚙️ Commands Executed:

📁 Update kubeconfig for EKS Cluster:

```
bash
CopyEdit
```

```
aws eks update-kubeconfig --region us-east-1 --name demo-cluster
```

```
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$ aws eks update-kubeconfig --region us-east-1 --name demo-cluster
Updated context arn:aws:eks:us-east-1:770535378489:cluster/demo-cluster in /home/cdickersoncloudcoder/.kube/config
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$
```

💡 This command generates or updates the kubeconfig file, which stores cluster access details. It enables `kubectl` to authenticate and manage the EKS cluster.

📁 Verify Cluster Services:

bash

CopyEdit

```
kubectl get svc
```

```
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$ aws eks update-kubeconfig --region us-east-1 --name demo-cluster
Updated context arn:aws:eks:us-east-1:770535378489:cluster/demo-cluster in /home/cdickersoncloudcoder/.kube/config
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$ kubectl get svc
NAME          TYPE          CLUSTER-IP   EXTERNAL-IP   PORT(S)    AGE
kubernetes    ClusterIP     10.100.0.1   <none>        443/TCP    44m
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$
```

💡 This command lists the services running in your Kubernetes cluster, verifying that you can interact with the cluster successfully.

🔍 What We Learned:

- We set up secure access to the EKS cluster using `aws eks update-kubeconfig`.

- We confirmed access by listing the running Kubernetes services with `kubectl get svc`.
- This step is essential for managing your cluster through Kubernetes commands.

✅ Summary:

You've just configured access to your Amazon EKS cluster and verified connectivity using `kubectl`. This adds to your understanding of how to securely interact with and manage your Kubernetes cluster on AWS.

📦 Step 3: 🔧 Managing Docker Permissions for Jenkins Integration

♦ Step Overview:

In this step, we managed Docker permissions on the Jenkins server to ensure Jenkins can interact with Docker containers. This is crucial for seamless integration between Jenkins and Docker, allowing Jenkins to build and deploy Dockerized applications.

⚙️ Commands Executed:

📁 Start Docker Container:

```
bash
CopyEdit
docker start jenkinscontainerid
```

💡 *This command starts the specified Docker container by its container ID.*

📁 Access Container with Root Permissions:

```
bash
CopyEdit
```

```
docker exec -u root -it jenkinscontainerid bash
```

```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker start [REDACTED]  
7aea760f26ee  
root@Jenkins-server:~# docker exec -u root -it [REDACTED] bash  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srw-rw---- 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/# chmod +x /var/run/docker.sock  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srwxrwx--x 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/#
```

💡 This allows you to enter the running Docker container with root access, enabling administrative actions.

📁 Inspect Docker Socket Permissions:

bash

CopyEdit

```
ls -l /var/run/docker.sock
```

```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker start [REDACTED]  
7aea760f26ee  
root@Jenkins-server:~# docker exec -u root -it [REDACTED] bash  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srw-rw---- 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/# chmod +x /var/run/docker.sock  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srwxrwx--x 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/#
```

💡 This reveals the current permissions of the Docker socket file, which controls Docker API access.

📁 Modify Docker Socket Permissions:

bash

CopyEdit

`chmod +x /var/run/docker.sock`

```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker start [REDACTED]  
7aea760f26ee  
root@Jenkins-server:~# docker exec -u root -it [REDACTED] bash  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srw-rw---- 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/# chmod +x /var/run/docker.sock  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srwxrwx--x 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/#
```

💡 This command grants execute permissions to the Docker socket, ensuring Jenkins can access Docker functionalities.

📁 Verify Updated Permissions:

bash

CopyEdit

`ls -l /var/run/docker.sock`

```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker start [REDACTED]  
7aea760f26ee  
root@Jenkins-server:~# docker exec -u root -it [REDACTED] bash  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srw-rw---- 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/# chmod +x /var/run/docker.sock  
root@7aea760f26ee:/# ls -l /var/run/docker.sock  
srwxrwx--x 1 root 112 0 May 11 16:29 /var/run/docker.sock  
root@7aea760f26ee:/#
```


💡 This confirms that the Docker socket now has execute permissions for the group, making it accessible for Jenkins.

📁 Access Jenkins Job URL: Jenkins AWS Multibranch Pipeline Job

S	W	Name	Last Success	Last Failure	Last Duration
❌	☁️	deploy-on-k8s	N/A	23 hr #14	4.3 sec

💡 This URL directs you to the Jenkins pipeline job, where you can monitor and manage your CI/CD workflows.

🔍 What We Learned:

- We managed Docker permissions to ensure Jenkins can interact with Docker containers.
- We confirmed permission changes using `chmod` and verified them with `ls -l`.
- We can now integrate Jenkins with Docker for automated builds and deployments.

✅ Summary:

You've just configured Docker permissions for seamless integration with Jenkins. This adds to your understanding of managing containerized applications and configuring secure access for automated pipelines.

📁 Step 4: 🚀 Installing and Configuring kubectl in Jenkins Container

♦ Step Overview:

In this step, we installed and configured `kubectl` within the Jenkins container. This setup is essential for managing Kubernetes clusters directly from the Jenkins environment, enabling automated deployments.

⚙️ Commands Executed:

📁 Access Jenkins Container with Root Permissions:

bash

CopyEdit

`docker exec -u root -it jenkinscontainerid bash`

```
root@Jenkins-server:~# docker exec -u root -it [REDACTED] bash
root@7aea760f26ee:/#
```

💡 This command provides root access to the Jenkins container, allowing administrative configurations.

📁 Download and Install kubectl:

bash

CopyEdit

```
curl -LO "https://storage.googleapis.com/kubernetes-release/release/$(curl -s
https://storage.googleapis.com/kubernetes-release/release/stable.txt)/bin/linu
x/amd64/kubectl"
chmod +x ./kubectl
mv ./kubectl /usr/local/bin/kubectl
```

```
root@7aea760f26ee:/# curl -LO "https://storage.googleapis.com/kubernetes-release/release/$(curl -s https://storage.googleapis.com/kubernetes-release/release/s
table.txt)/bin/linux/amd64/kubectl"
chmod +x ./kubectl
mv ./kubectl /usr/local/bin/kubectl
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left  Speed
100 53.7M  100 53.7M    0     0  46.5M      0  0:00:01  0:00:01 --:--:-- 46.5M
root@7aea760f26ee:/#
```

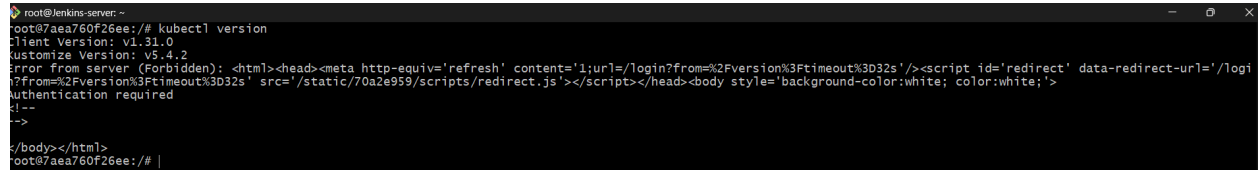
💡 These commands download the latest version of **kubectl**, make it executable, and move it to the system path for global access.

📁 Verify kubectl Installation:

bash

CopyEdit

kubectl version



```
root@jenkins-server: ~  
root@7aea760f26ee:/# kubectl version  
Client Version: v1.31.0  
Kustomize Version: v5.4.2  
Error from server (Forbidden): <html><head><meta http-equiv='refresh' content='1;url=/login?from=%2Fversion%3Ftimeout%3D32s' /><script id='redirect' data-redirect-url='/login?from=%2Fversion%3Ftimeout%3D32s' src= /static/70a2e959/scripts/redirect.js /></script></head><body style='background-color:white; color:white;'>  
Authentication required  
</body></html>  
root@7aea760f26ee:/#
```

💡 This confirms that **kubectl** is installed correctly and displays the client version.

📁 Check AWS IAM Authenticator (Optional):

bash

CopyEdit

aws-iam-authenticator help

```
root@jenkins-server: ~
root@7aea760f26ee:/# aws-iam-authenticator help
A tool to authenticate to Kubernetes using AWS IAM credentials

Usage:
  aws-iam-authenticator [command]

Available Commands:
  add          add IAM entity to an existing aws-auth configmap
  completion   Generate the autocompletion script for the specified shell
  help         Help about any command
  init         Pre-generate certificate, private key, and kubeconfig files for the server.
  server       Run a webhook validation server suitable that validates tokens using AWS IAM
  token        Authenticate using AWS IAM and get token for Kubernetes
  verify       Verify a token for debugging purpose
  version      Version will output the current build information

Flags:
  -i, --cluster-id ID          Specify the cluster ID, a unique-per-cluster identifier for your aws-iam-authenticator installation.
  -c, --config filename       Load configuration from filename
  --feature-gates mapStringBool A set of key=value pairs that describe feature gates for alpha/experimental features. Options are:
                               AllAlpha=true|false (ALPHA - default=false)
                               AllBeta=true|false (BETA - default=false)
                               ConfiguredInitDirectories=true|false (ALPHA - default=false)
                               IAMIdentityMappingCRD=true|false (ALPHA - default=false)
  -h, --help                  help for aws-iam-authenticator
  -l, --log-format string      Specify log format to use when logging to stderr [text or json] (default "text")

Use "aws-iam-authenticator [command] --help" for more information about a command.
root@7aea760f26ee:/#
```

 This verifies if `aws-iam-authenticator` is available for Kubernetes authentication, useful for secure cluster access.

What We Learned:

- We installed `kubectl` inside the Jenkins container, enabling Kubernetes management.
- We ensured the binary was executable and globally accessible.
- We verified the installation and explored `aws-iam-authenticator` for secure access.

Summary:

You've just installed and configured `kubectl` in the Jenkins container, making it possible to manage Kubernetes clusters directly from Jenkins. This adds to your understanding of integrating Kubernetes with CI/CD pipelines.

Step 5: Configuring Kubernetes Authentication with AWS IAM in Jenkins

♦ Step Overview:

In this step, we configured Kubernetes authentication in the Jenkins container using AWS IAM, allowing secure access to the Amazon EKS cluster. This setup ensures that the Jenkins container can authenticate and manage the EKS cluster securely.

Commands Executed:

Create or Edit Kubernetes Configuration File:

bash

CopyEdit

nano config

```
root@jenkins-server:~#  
root@jenkins-server:~# nano config
```

 This command opens a new or existing configuration file for Kubernetes, where authentication details will be specified.

Configuration File Content:

yaml

CopyEdit

```
apiVersion: v1  
kind: Config  
clusters:  
- cluster:  
  certificate-authority-data: <certificate-data>  
  server: <endpoint>  
  name: kubernetes  
contexts:
```

```

- context:
  cluster: kubernetes
  user: aws
  name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - <cluster-name>

```

```

root@jenkins-server: ~
GNU nano 8.1
apiVersion: v1
kind: Config
clusters:
- cluster:
  certificate-authority-data: <certificate-data>
  server: <endpoint-url>
  name: kubernetes
contexts:
- context:
  cluster: kubernetes
  user: aws
  name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - <cluster-name>

```

💡 This configuration enables secure access to the EKS cluster using AWS IAM authentication, where **<certificate-data>** is the encoded certificate and **<endpoint>** is the EKS API URL.

📁 EKS Cluster Endpoint Details:

Cluster API Endpoint:

bash
CopyEdit

```
certificate-authority-data:
server: <endpoint>|
name: kubernetes
```

server: https://952#####.gr7.us-east-1.eks.amazonaws.com

```
certificate-authority-data: <certificate-data>
server: https://952#####630F53.gr7.us-east-1.eks.amazonaws.com
name: kubernetes
```

Details

API server endpoint

 https://952#####630F53.gr7.us-east-1.eks.amazonaws.com

-

Cluster Name:

bash

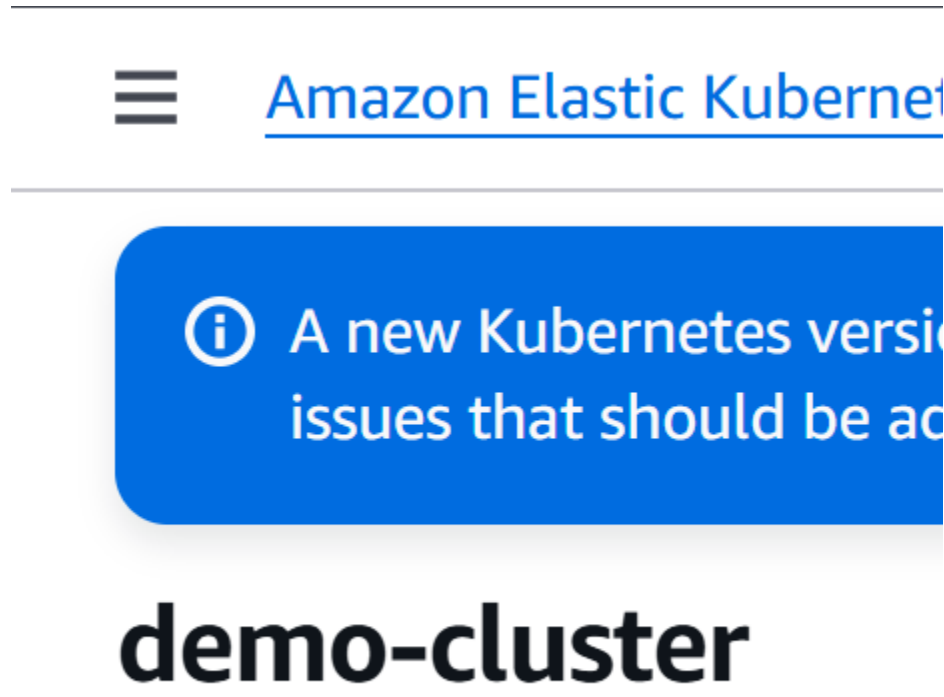
CopyEdit

- demo-cluster

```
- token
- "-j"
- demo-cluster
```

•

📁 Access EKS Cluster Console:
[EKS Cluster Console](#)



💡 This link provides direct access to the EKS console for reviewing cluster configurations.

📁 Inspect Existing Kubernetes Configuration:

bash

CopyEdit

cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop\$ cat ~/.kube/config

```
cdickersoncloudcoder@BeastMode:/mnt/c/Users/19168/Desktop$ cat ~/.kube/config
apiVersion: v1
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tck11SURCVENDQWUyZDZ3SUZBZ01JU2pzMw1YY29ZNR3RFFZSkVwWk10dmNoQVFFTEJRQXdGVEVUTUJFR0ExVUUKQ...
    server: https://252.1.1.1:443
  name: aws:eks:us-east-1:77853578489:cluster/demo-cluster
certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FUR50tLS0tck11SURCVENDQWUyZDZ3SUZBZ01JU2pzMw1YY29ZNR3RFFZSkVwWk10dmNoQVFFTEJRQXdGVEVUTUJFR0ExVUUKQ...
```

💡 This command displays the current Kubernetes configuration file for the local user.

📁 Sample Config Content Displayed:

yaml

CopyEdit


```

apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data:
LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ01JU2pzMW1YY29ZNj
R3RFFZSk1vWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQ>
    server: https://952#####630F53.gr7.us-east-1.eks.amazonaws.com
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - demo-cluster

```

```

GNU nano 8.1 config
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1CRUdJTiBDRVJUSUZJQ0FURS0tLS0tCk1JSURCVENDQWUyZ0F3SUJBZ01JU2pzMW1YY29ZNjR3RFFZSk1vWklodmNOQVFFTEJRQXdGVEVUTUJFR0ExVUUKQ>
    server: https://952#####630F53.gr7.us-east-1.eks.amazonaws.com
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
    name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - demo-cluster

```

💡 This reveals the full kubeconfig setup, including the cluster API endpoint and the AWS IAM authenticator configuration.

Verify the Kubernetes Config in Jenkins Container:

bash

CopyEdit

```
root@Jenkins-server:~# cat config
```

```

root@Jenkins-server: ~
root@Jenkins-server:~# cat config
apiVersion: v1
kind: Config
clusters:
- cluster:
    certificate-authority-data: LS0tLS1lbnN2EzVmIawEp1wIhSbGN6QWVGdzB5TlRB...
    hNS2EzVmIawEp1wIhSbGN6QWVGdzB5TlRB...
    QKR3QXdnZ0VLCKFvSUJBUUNycIkwVmhGF...
    lvt2xaVwZ2NzFSQjVUcGZLNjBncwp5I...
    MlJOYzFKRVZPVE1ONDJhNDdRMGTRE...
    JpWEtkekNoQWdNQkFBR2pXVEJYTUEOR0E...
    akFNZ2dwcmlRXSmxjbTVsZEdwek1BME...
    QvCnd3SDhOS1pTY2JNakNsTWZorGpFbDF...
    cw5LRlY5RjJpWE1qbApYRU1taGRXZVZW...
    JiLOW2SVU20Fc2cFZyem5OTFBaOHg0NXkKRKJKd55...
    server: https://952...61CB729630F53.gr7.us-east-1.eks.amazonaws.com
    name: kubernetes
contexts:
- context:
    cluster: kubernetes
    user: aws
  name: aws
current-context: aws
users:
- name: aws
  user:
    exec:
      apiVersion: client.authentication.k8s.io/v1beta1
      command: /usr/local/bin/aws-iam-authenticator
      args:
        - "token"
        - "-i"
        - demo-cluster

```

Details

 [https://95\[REDACTED\]630F53.gr7.us-east-1.eks.amazonaws.com](https://95[REDACTED]630F53.gr7.us-east-1.eks.amazonaws.com)

What We Learned:

-  **Summary:**

You've just set up Kubernetes authentication using AWS IAM in the Jenkins container, providing secure access to the EKS cluster. This adds to your understanding of securely managing Kubernetes clusters using AWS authentication.

Step 6: Replacing and Verifying Kubernetes Config in Jenkins Container

♦ Step Overview:

In this step, we replaced the existing Kubernetes configuration file in the Jenkins container and verified the changes. This ensures that the Jenkins container has the correct kubeconfig file for secure Kubernetes access.

Commands Executed:

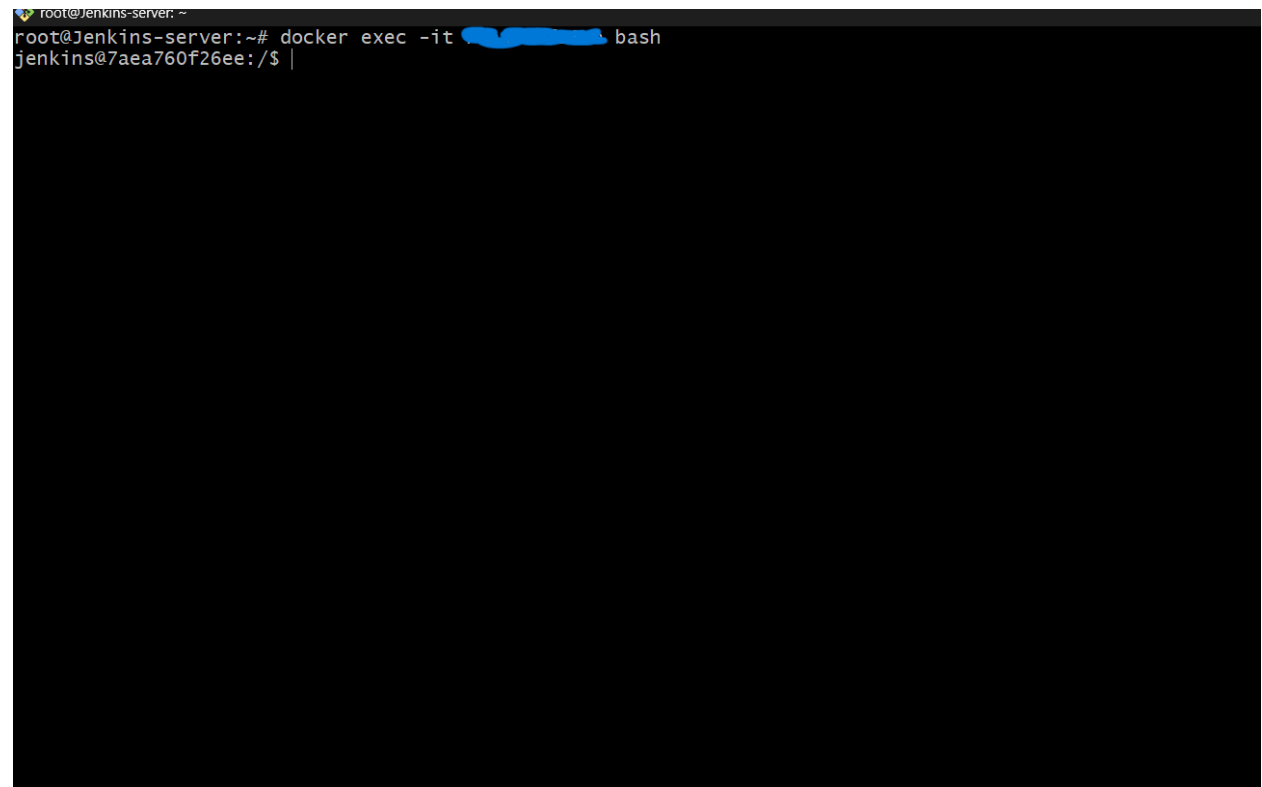
Access Jenkins Container with Interactive Shell:

bash

CopyEdit

```
root@Jenkins-server:~# docker exec -it ##### bash
```

```
jenkins@#####:/$
```



```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker exec -it [redacted] bash  
jenkins@7aea760f26ee:/$ |
```

 *This command provides interactive access to the Jenkins container for direct configuration management.*

Navigate to Home Directory and Confirm Location:

bash

CopyEdit

```
jenkins@#####:/$ cd ~
```

```
jenkins@#####:~$ pwd
```

/var/jenkins_home

```
jenkins@jenkins:~$ cd ~
jenkins@jenkins:~$ pwd
/var/jenkins_home
jenkins@jenkins:~$
```

💡 This confirms that we are in the Jenkins home directory (*/var/jenkins_home*).

📁 **Attempt to Create .kube Directory:**

bash

CopyEdit

```
jenkins@jenkins:~$ mkdir .kube
```

mkdir: cannot create directory '.kube': File exists

```
root@Jenkins-server: ~  
jenkins@jenkins-server:~$ mkdir .kube  
mkdir: cannot create directory '.kube': File exists  
jenkins@jenkins-server:~$ |
```

💡 The command fails because the `.kube` directory already exists, which is the default location for Kubernetes configurations.

📁 Copy the Kubernetes Configuration File to Jenkins Container:

bash

CopyEdit

```
root@Jenkins-server:~# docker cp config #####:/var/jenkins_home/.kube/
```

Successfully copied 4.1kB to #####:/var/jenkins_home/.kube/

```
root@Jenkins-server: ~  
root@Jenkins-server:~# docker cp config #####:/var/jenkins_home/.kube/  
Successfully copied 4.1kB to #####:/var/jenkins_home/.kube/  
root@Jenkins-server:~#
```

💡 This command securely copies the Kubernetes configuration file we created in Step 5 into the `.kube` directory of the Jenkins container.

📁 Access Jenkins Container Again to Verify:

```
bash  
CopyEdit  
root@Jenkins-server:~# docker exec -it ##### bash  
jenkins@#####:/$ cd ~
```

💡 This provides another interactive session for verifying the configuration.

📁 Verify the Kubernetes Config File Properties:

```
bash  
CopyEdit  
jenkins@#####:~$ ls -l /var/jenkins_home/.kube/config  
-rw-r--r-- 1 root root 2120 May 11 18:54 /var/jenkins_home/.kube/config
```

💡 This confirms the file size, permissions, and ownership, indicating a successful file copy.

📁 View the Kubernetes Configuration File Content:

In

-

Branch Sources

Git

Project Repository ?

https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-

-

bas

Co

cd

 $\sim /$

pla

CopyEdit

[default]

```
aws_access_key_id = A*****7W
```

```
aws_secret_access_key = y*****g
```

⚠ Sensitive Information Notice:

The AWS Access Key and Secret Access Key are sensitive credentials. Ensure that these keys are secured and not exposed publicly.

📁 Configuring AWS Credentials in Jenkins:

- [🔗](#) **Access the AWS Credentials Configuration:**
AWS Credentials Configuration - Jenkins

Dashboard > aws-multibranch-pipeline > Credentials > Folder > Global credentials (unrestricted) >

New credentials

Kind

Secret text

Secret

.....

ID ?

jenkins_aws_access_key_id

Description ?

- [🔗](#) **Manage Credentials in the Folder:**
Manage AWS Credentials in Jenkins

New credentials

Kind

Secret text

Secret

.....

ID ?

jenkins_aws_secret_access_key

Description ?

Create

-  **Create New Credentials in Jenkins:**
Add New Credentials in Jenkins
-  **Configure AWS Credentials:**
 - **Kind:** AWS Credentials
 - **Access Key ID:** A*****7W
 - **Secret Access Key:** y*****g
 - **ID (optional):** aws-credentials



Jenkins



1



Chad Dickerson



log c

Global credentials (unrestricted)

+ Add Credentials

Credentials that should be available irrespective of domain specification to requirements matching.

ID	Name	Kind	Description
 ec2-server-key	ec2-user	SSH Username with private key	
 jenkins_aws_access_key_id	jenkins_aws_access_key_id	Secret text	
 jenkins_aws_secret_access_key	jenkins_aws_secret_access_key	Secret text	

Icon: S M L

💡 *Storing AWS credentials in Jenkins allows secure access to AWS services for EKS deployment without exposing the keys directly in scripts.*

🔍 What We Learned:

- We reviewed local AWS credentials to ensure they are accurate.
- We securely added AWS credentials to Jenkins, allowing it to access AWS resources for EKS deployment.
- We ensured that the AWS credentials are protected by storing them in Jenkins credentials management.

✅ Summary:

You have securely configured AWS credentials in Jenkins, allowing your Jenkins pipeline to interact with AWS services, including deploying and managing an Amazon EKS cluster. This is a crucial step for secure and automated cloud operations.

📁 Step 8: Automating Deployment with Jenkins Pipeline for EKS

♦ Step Overview:

In this step, we set up an automated Jenkins pipeline to deploy a Docker container to AWS EKS using `kubectl`. This provided insight into integrating Jenkins with AWS EKS and using Kubernetes for container orchestration.

⚙️ Commands Executed:

📁 Jenkins Pipeline Script:

bash

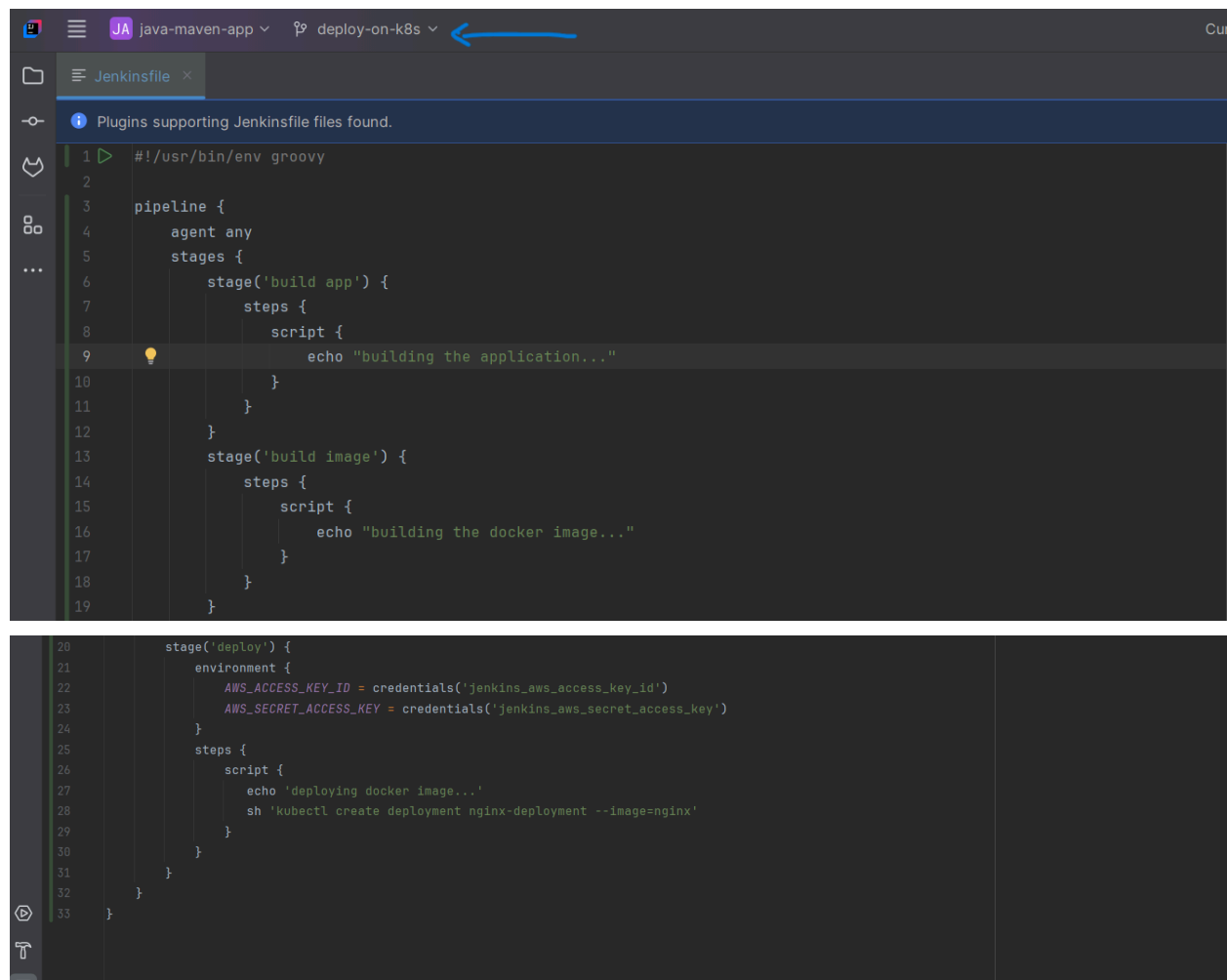
CopyEdit

```
#!/usr/bin/env groovy
```

```
pipeline {
  agent any
  stages {
    stage('build app') {
      steps {
        script {
          echo "building the application..."
        }
      }
    }
  }
  stage('build image') {
```

```
        steps {
            script {
                echo "building the docker image..."
            }
        }
    }
    stage('deploy') {
        environment {
            AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
            AWS_SECRET_ACCESS_KEY =
credentials('jenkins_aws_secret_access_key')
        }
        steps {
            script {
                echo 'deploying docker image...'
                sh 'kubectl create deployment nginx-deployment
--image=nginx'
            }
        }
    }
}
```

}



The screenshot shows a Jenkinsfile editor with a dark theme. The top bar indicates the project is 'java-maven-app' and the pipeline is 'deploy-on-k8s'. A blue arrow points to the pipeline name. The Jenkinsfile content is as follows:

```
1  #!/usr/bin/env groovy
2
3  pipeline {
4      agent any
5      stages {
6          stage('build app') {
7              steps {
8                  script {
9                      echo "building the application..."
10                 }
11             }
12         }
13         stage('build image') {
14             steps {
15                 script {
16                     echo "building the docker image..."
17                 }
18             }
19         }
20     }
21     stage('deploy') {
22         environment {
23             AWS_ACCESS_KEY_ID = credentials('jenkins_aws_access_key_id')
24             AWS_SECRET_ACCESS_KEY = credentials('jenkins_aws_secret_access_key')
25         }
26         steps {
27             script {
28                 echo 'deploying docker image...'
29                 sh 'kubectl create deployment nginx-deployment --image=nginx'
30             }
31         }
32     }
33 }
```

💡 Explanation:

This script defines the Jenkins pipeline stages for building the app, building the Docker image, and deploying it to a Kubernetes cluster on AWS EKS using **kubectl**. The AWS credentials are securely stored and used within the deployment stage.

📁 Jenkins Job Configuration URL:

bash
CopyEdit

<http://#####/job/aws-multibranch-pipeline/configure>



Filter by name (with regular expression)

Regular expression ?

deploy-on-k8s

💡 Explanation:

This URL directs you to the Jenkins job configuration page. You can use it to customize and configure the pipeline job for your project.

📁 Git Commands for Version Control:

bash

CopyEdit

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git add .

bash

CopyEdit

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git commit -m "add kubctl deploy"

[deploy-on-k8s d286a99] add kubctl deploy
1 file changed, 33 insertions(+)

bash

CopyEdit

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git push

```
C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git add .

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git commit -m "add kubctl deploy"
[deploy-on-k8s d286a99] add kubctl deploy
1 file changed, 33 insertions(+)

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>git push
Enumerating objects: 5, done.
Counting objects: 100% (5/5), done.
Delta compression using up to 16 threads
Compressing objects: 100% (3/3), done.
Writing objects: 100% (3/3), 556 bytes | 556.00 KiB/s, done.
Total 3 (delta 1), reused 0 (delta 0), pack-reused 0
remote:
remote: To create a merge request for deploy-on-k8s, visit:
remote:   https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-app/-/merge_requests/new?merge_request%5Bsource_branch%5D=deploy-on-k8s
remote:
To https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-app.git
   7a9a052..d286a99  deploy-on-k8s -> deploy-on-k8s

C:\Users\19168\Desktop\DevOps Bootcamp\Entering Learning Phase 3 - DevOps Core\11 - Kubernetes on AWS - EKS\java-maven-app>
```

💡 **Explanation:**

We committed and pushed the changes to the Git repository. This ensures that the pipeline has the latest code, which can then be automatically deployed during the next Jenkins job run.

📁 **GitLab Merge Request URL:**

bash

CopyEdit

remote:

remote: To create a merge request for deploy-on-k8s, visit:

remote:

https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-app/-/merge_requests/new?merge_request%5Bsource_branch%5D=deploy-on-k8s

💡 **Explanation:**

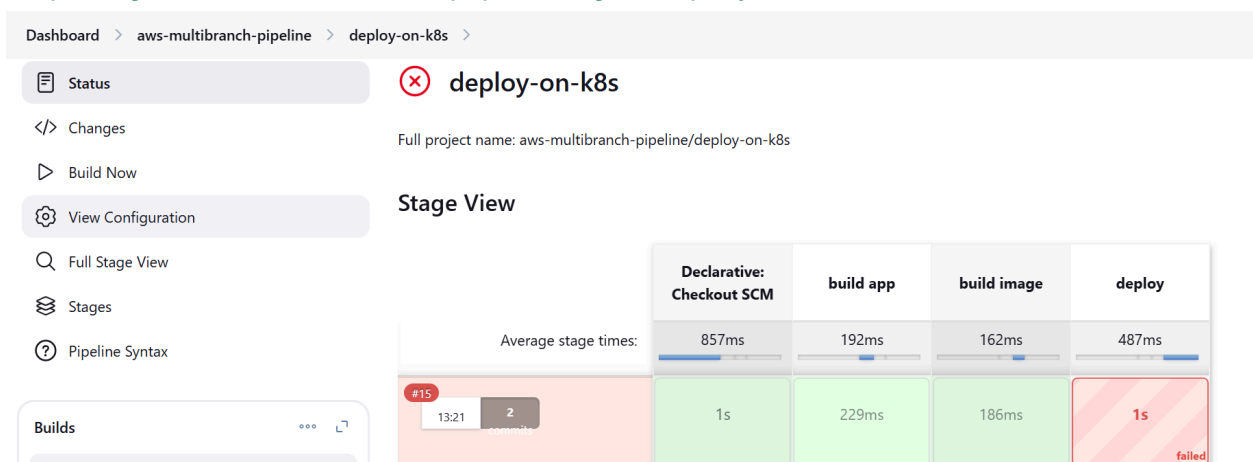
This is the GitLab URL for creating a merge request to merge your `deploy-on-k8s` branch into the main branch. It's part of the collaboration process to review and approve changes before merging them into the primary codebase.

📁 **Jenkins Pipeline Job URL:**

bash

CopyEdit

<http://job/aws-multibranch-pipeline/job/deploy-on-k8s/>



Dashboard > aws-multibranch-pipeline > deploy-on-k8s > #16

```
Deploying docker image...
[Pipeline] sh
+ kubectl create deployment nginx-deployment --image=nginx
error: failed to create deployment: Unauthorized
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE
```

Started by user [Chad Dickerson](#)

```
> git rev-parse --resolve-git-dir
/var/jenkins_home/caches/git-486efe89efd6d99cba4bc9dc9c817a79/.git #
timeout=10
Setting origin to
https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-app.git
> git config remote.origin.url
https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops-core/11-k8s-on-aws-eks/java-maven-app.git # timeout=10
Fetching origin...
Fetching upstream changes from origin
> git --version # timeout=10
> git --version # 'git version 2.39.5'
> git config --get remote.origin.url # timeout=10
using GIT_ASKPASS to set credentials
> git fetch --tags --force --progress -- origin
+refs/heads/*:refs/remotes/origin/* # timeout=10
Seen branch in repository origin/deploy-on-k8s
Seen branch in repository origin/deploy-to-lke
Seen branch in repository origin/feature/payment
Seen branch in repository origin/jenkins-jobs
Seen branch in repository origin/jenkins-shared-lib
Seen branch in repository origin/master
Seen branch in repository origin/starting-code
Seen 7 remote branches
Obtained Jenkinsfile from d286a9964469e4dc07b5d9194dd4ac1ea219aa1d
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins
```

```

in /var/jenkins_home/workspace/ltibranch-pipeline_deploy-on-k8s
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Declarative: Checkout SCM)
[Pipeline] checkout
The recommended git tool is: git
using credential gitlab-credentials
  > git rev-parse --resolve-git-dir
/var/jenkins_home/workspace/ltibranch-pipeline_deploy-on-k8s/.git #
timeout=10
Fetching changes from the remote Git repository
  > git config remote.origin.url
https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops
-core/11-k8s-on-aws-eks/java-maven-app.git # timeout=10
Fetching without tags
Fetching upstream changes from
https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops
-core/11-k8s-on-aws-eks/java-maven-app.git
  > git --version # timeout=10
  > git --version # 'git version 2.39.5'
using GIT_ASKPASS to set credentials
  > git fetch --no-tags --force --progress --
https://gitlab.com/devops-bootcamp8333314/entering-learning-phase-3-devops
-core/11-k8s-on-aws-eks/java-maven-app.git
+refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision d286a9964469e4dc07b5d9194dd4ac1ea219aa1d
(deploy-on-k8s)
  > git config core.sparsecheckout # timeout=10
  > git checkout -f d286a9964469e4dc07b5d9194dd4ac1ea219aa1d # timeout=10
Commit message: "add kubctl deploy"
  > git rev-list --no-walk d286a9964469e4dc07b5d9194dd4ac1ea219aa1d #
timeout=10
[Pipeline] }
[Pipeline] // stage
[Pipeline] withEnv
[Pipeline] {
[Pipeline] stage
[Pipeline] { (build app)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
building the application...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage

```

```

[Pipeline] { (build image)
[Pipeline] script
[Pipeline] {
[Pipeline] echo
building the docker image...
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (deploy)
[Pipeline] withCredentials
Masking supported pattern matches of $AWS_ACCESS_KEY_ID or
$AWS_SECRET_ACCESS_KEY
[Pipeline] {
[Pipeline] script
[Pipeline] {
[Pipeline] echo
deploying docker image...
[Pipeline] sh
+ kubectl create deployment nginx-deployment --image=nginx
error: failed to create deployment: Unauthorized
[Pipeline] }
[Pipeline] // script
[Pipeline] }
[Pipeline] // withCredentials
[Pipeline] }
[Pipeline] // stage
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
ERROR: script returned exit code 1
Finished: FAILURE

```

Explanation:

This URL leads to the specific Jenkins pipeline job for [deploy-on-k8s](#). You can use this link to monitor the status of your deployment pipeline and view the logs for each stage.

What We Learned:

- Gained understanding of automating deployments with Jenkins and Kubernetes (EKS).

- Integrated Jenkins with AWS EKS for containerized application deployment.
- Leveraged Jenkins pipelines to streamline build and deployment processes.

✅ **Summary:**

You've just automated the deployment of a Docker container to AWS EKS using a Jenkins pipeline. This adds to your understanding of cloud-native deployment automation and how to integrate Jenkins with Kubernetes for efficient DevOps workflows. 🚀